

Mini Project 3 Report

By Emily Herman, Megan Ruiz, Erik Ellams, Nickolas Diaz

Language specification	1
BNF	1
Set!	1
AST definition	2
Desugarer.....	2
Interpreter	3
Test cases	3
Reflection.....	3

Language specification

A document describing the grammar of your extended language, listing all surface forms and all core forms and explaining which constructs are sugar and which are core.

BNF

```
1. <expr> ::= <num>
2. | { if <expr> <expr> <expr> }
3. | { lam <var> <expr> }
4. | { let1 { <var> <expr> } <expr> }
5. | { set! <var> {<expr> } }
6. | { <expr> <expr> }
7. | { and <expr> <expr> }
8. | { or <expr> <expr> }
9. | <var>
10. | {begin +<expr> }
11. | { while <expr> <expr> }
```

Set!

The set! functionality evaluates the expression e, finds the location of the variable x in the environment, and updates the store so that location now holds the new value. The variable x must already exist. If it does not, the interpreter should report an error.

Example Usage: {set! variable_name expersion}

For the BNF it takes the form of `set! <var> <expr>`, it is like `let1` but it does not have an appending `<expr>` returned. Meaning that using it means that it cannot chain set expressions by itself.

Begin

The `begin` evaluates a list of expressions from left to right and returns the value of the last one. The intermediate values are discarded, but their side effects (such as assignments) persist in the store.

Example Usage: `{begin expr1 expr2 ... exprN}`

The BNF consists of `{begin +<expr> }`, where `+<expr>` means 1 or many expressions, as noted in the `+` symbol. This is because it can evaluate 1 or infinite expressions. The single `1` is needed as one must be the last to return its value.

While

The `while` loop takes in a condition and a body, where if the condition is true the body is running else it goes to the next condition. When the body is finished, it goes back to the condition, forming a loop.

Example Usage: `{while cond body}`

The BNF consists of `{while <expr> <expr> }`. Since we don't have a proper condition BNF syntax we will have to just use the `<expr>` symbol for it. The same goes for the body.

AST definition

The Abstract Syntax Tree (AST) for this interpreter is defined using a recursive `Exp` data type that distinguishes between core logic and syntactic sugar. The base cases include `numE` for literal values and `varE` for variable identifiers. To support stateful programming we added `set1E` for variable mutation and `beginE` to handle sequences of expressions. The `whileE` constructor was added to support iteration alongside `andE` and `orE` for boolean logic. While the interpreter is required to recognize all these constructors to pass Plait's checking, the sugar constructs are designed to be eliminated by the desugarer before reaching the evaluation stage.

Desugarer

The desugarer acts as a pre-processor that simplifies the AST by translating complex "sugar" nodes into combinations of core "primitive" nodes. For example, "and" and "or" are transformed into nested "if" expressions. The most significant transformation is the `while` loop rule. A `whileE` is rewritten as an `ifE`, where the "true" branch contains a `beginE`. This begins block executes in the loop body and then recursively calls the `while` loop again.

while the "false" branch returns a dummy value of 0. This recursive expansion allows us to implement loops and complex logic gates without adding any new complexity to the core interpreter logic.

Interpreter

The interpreter implements a decoupled Environment and Store model to manage state and memory. In this architecture, the Environment does not map names directly to values, instead it maps names to memory addresses which are called "Locations". These "Locations" then point to the Store, represented by a global MEMORY vector, where the actual data resides.

When the interpreter encounters a let1E, it allocates a new address in the MEMORY vector via the create-num helper and binds the variable name to that specific address in the environment. When a variable is accessed via varE the interpreter performs a "two-step lookup". Firstly, it finds the address in the environment and then secondly begins reading the value from that address in the Store. This separation is critical for the "set!" (set1E) functionality. It allows us to update the value at a specific memory location without changing the Environment bindings. Also, any closure or function sharing that environment will immediately "see" the updated value enabling true mutable state and shared memory across different scopes.

Test cases

At least five test programs demonstrate mutation, sequencing, the while loop, closures interacting with mutation, and a program where evaluation order affects the result. What was the hardest part of moving from substitution to the environment and store model?

Reflection

- 1) Why does the interpreter now need to return a store alongside a value?

The interpreter was modified to store the location of a variable instead of the previous value. Example is variable name -> address location -> variable value. The previous dictionary/map now stores the variable name as key and address location as the value. A

vector/list stores the value of the variable, and its index is the address of the location. This more accurately represents how interpreted languages work.

2) What would break if you forgot to thread the store through a begin expression?

It will probably be one of many things:

The type checker will make an error saying that the store is not there

The expressions in the will not update the store

The expressions will not execute or be skipped

3) Why does the while loop not a core construct?

The while loop can be broken down into the already existing core constructs. It can be desugared into a condition, begin and body expressions, like.

```
{cond
```

```
{begin body {while cond body}}
```

```
0}
```

By moving the while loop to the desugarer we will make the code much simpler as we will not need to touch the interpreter.